

Тема 2. Подредени множества и низове.

Операции с низове. Регулярни множества и изрази

В компютърните науки под низ се разбира крайна поредица от символи, чиито елементи и дължина могат да се променят. Низовете се разглеждат като тип данни и често са във вид на масиви от байтове (или думи), които съхраняват последователност от елементи, използвайки кодиране на символите.

Според езика за програмиране и използвания тип данни променлива, декларирана като низ се съхранява в паметта:

- **Статично** - с фиксиране максимална дължина;
- **Динамично** - с различен брой елементи.

Във формалните езици, които се използват в математическата логика и теоретичната информатика, низ е крайна последователност от символи от дадено множество, наречено азбука.

Тема 2. Подредени множества и низове.

Операции с низове. Регулярни множества и изрази

1. Подредени множества

Множества, на които елементите са подредени в ред (низ или дума). За тях съществуват понятия като първи, последен, предходен, следващ елемент и т.н.

- Азбука – произволно крайно множество от символи.

Обозначение: $\Sigma = \{a, b, v, \dots y\}$

- Низ – (или дума) над Σ се нарича всяка крайна редица от букви от азбуката.

Обозначения:

$\Lambda (\epsilon)$ - дума, която няма букви (празна дума)

a, b, c – с началните букви от латинската азбука се бележат **символи**, букви от Σ

t, u, v – с крайните букви от латинската азбука се бележат **символни низове**, думи

A, B, C – с главни латински букви се бележат **множества на низове**

$||$ - дължина на низ (дума)

Σ^* – множество от всички възможни низове включително и празен низ Λ

Σ^+ – множество от всички възможни низове без празен низ Λ

Σ (Сигма) Λ (Ламбда)

Тема 2. Подредени множества и низове.

Операции с низове. Регулярни множества и изрази

Примери:

$$\Sigma = \{a, b\}$$

$$\Sigma^* = \{\Lambda, a, b, ab, ba, aa, bb, \dots\}$$

$$\Sigma^+ = \{a, b, ab, ba, aa, bb, \dots\}$$

$\{\} = \emptyset$ празно множество от низове

$\{\Lambda\}$ - множество, което съдържа празен низ като елемент.

Това множество не е празно.

2. Операции с низове:

- Съединение;
- Обединение;
- Затворена обвивка.

Тема 2. Подредени множества и низове.

Операции с низове. Регулярни множества и изрази

2.1. Съединение (конкатенация) на низове и множества от низове:

Нека са дадени низове u и w , които принадлежат на множествата U и W съответно:

$$u \in U \quad w \in W$$

Множеството UW , получено чрез операция съединение на множествата U и W съдържа елементите x .

$$UW = \{x \mid x=uw, u \in U \quad w \in W\}$$

Всяка дума от UW се получава от съединяване на дума от U с дума от W .

Пример: Нека са зададени следните множества: $U = \{a, ab, aab\}$ и $W = \{b, bb\}$

Резултатът от канкатенация на WU е: $WU = \{ba, bba, bab, bbab, baab, bbaab\}$

Операцията конкатенация е некомутативна, но е асоциативна:

$$UW \neq WU$$

$$u(vw) = (uv)w, \text{ където } u, w, v \in \Sigma^*$$

Тема 2. Подредени множества и низове.

Операции с низове. Регулярни множества и изрази

2.2. Обединение на низове

Пример:

$U = \{a, ab, abb\}$ и $W = \{b, bb\}$ Резултата е: $U \cup W = \{a, ab, aab, b, bb\}$

т.е. към елемент от U се добавят елементи от W

2.3. Затворена обвивка S^* (итерация, действие звезда)

Резултатът от операцията представлява множество, състоящо се от празната дума и всички думи, образувани чрез съединяване на краен брой думи от S .

$\Sigma^* \leftrightarrow S^*$ - съединение на букви (низове) от множеството S .

$S^* = S^0 \cup S^1 \cup S^2, \text{ като } S^0 = \{\Lambda\}$

$S^i = S^{i+1}S$ за всяко $i > 0$

Пример:

$S = \{ab, bb\} \rightarrow S^* = \{\Lambda, ab, bb, abab, abbb, \dots\}$

Тема 2. Подредени множества и низове.

Операции с низове. Регулярни множества и изрази

3. Регулярни множества

Регулярни множества са клас множества от думи на Σ . Те се получават чрез краен брой прилагания на следните правила:

1. Всяко крайно множество на думи над Σ включително и празното е регулярно.
2. Ако U и V са регулярни множества, то тяхното обединение $U+V$ или конкатенация UV са също регулярни множества.
3. Ако S е регулярно множество над Σ , то и S^* е също регулярно.

Пример:

Множество от думи на Σ , което съдържа думи, само с aa и/или bb е регулярно.

$T = \{aa, aab, abb, abba, baab, \dots\}$

$T = \{a^n b^n \mid n > 0\}$ съдържа n „а“ и след това n „b“. Това не е регулярно множество, защото не е крайно.

Тема 2. Подредени множества и низове.

Операции с низове. Регулярни множества и изрази

4. Регулярни изрази

Регулярни изрази са средства за представяне или запис на регулярни множества, което дава възможност за алгебрично изчисляване с регулярни множества.

Регулярни изрази над азбуката се дефинират рекурсивно по следния начин:

1. Празната дума и празното множество от думи са регулярни изрази.
2. Всяка буква (σ) от Σ е регулярен израз, $\sigma \in \Sigma$.
3. Ако P и Q са регулярни изрази, то $(P+Q)$, (PQ) , $(P)^*$ са регулярни изрази.

Приоритет на операциите:

(U) — изрази в скоби имат най — висок приоритет

U^* — затворена обвивка

UV —конкатенация

$U+V$ —обединение

Тема 2. Подредени множества и низове.

Операции с низове. Регулярни множества и изрази

Примери:

$\Sigma = \{a, b\} \rightarrow ((a + ba))^*$ – регулярен израз върху азбуката

ab^* – регулярен израз, описващ множеството от всички думи над Σ , започващи с a

$a^*ba^*ba^*$ е регулярен израз, описващ множеството от всички думи над Σ , започващи с s и завършващи с $2b$

5. Изчисления с регулярни изрази. Формули за еквивалентност на регулярни изрази

Нека R, S, T са произволни множества над Σ .

Всеки регулярен израз R описва множество $\tilde{R}$ от думи над Σ , т.е. $\tilde{R} \subseteq \Sigma^*$

$R = \Lambda \rightarrow \tilde{R} = \{\Lambda\}$ – множество, състоящо се от празна дума

$R = \emptyset \rightarrow \tilde{R} = \{\emptyset\}$ – празно множество, което не съдържа никакви думи

$R = \sigma \rightarrow \tilde{R} = \{\sigma\}$ – множество, състоящо се от буквата σ

Тема 2. Подредени множества и низове.

Операции с низове. Регулярни множества и изрази

5. Изчисления с регулярни изрази. Формули за еквивалентност на регулярни изрази

Нека R_1 и R_2 са регулярни изрази, които описват $\widetilde{R}_1$ и $\widetilde{R}_2$:

- $\tilde{R} = \widetilde{R}_1 \cdot \widetilde{R}_2 = \{r_1 r_2 \mid r_1 \in \widetilde{R}_1, r_2 \in \widetilde{R}_2\}$ – конкатенация
- $R = R_1 + R_2 \rightarrow \tilde{R} = \{x \mid x \in \widetilde{R}_1, x \in \widetilde{R}_2\}$ – обединение
- $R = (\widetilde{R}_1)^* \rightarrow$ множеството, образувано чрез операция звезда върху елемента на множеството $\widetilde{R}_1$

Тема 2. Подредени множества и низове.

Операции с низове. Регулярни множества и изрази

За произволни регулярни изрази R , S и T са валидни следните формули за еквивалентност:

$$1. R + S = S + R;$$

$$R + \emptyset = \emptyset + R;$$

$$R + R = R;$$

$$(R + S) + T = R + (S + T)$$

$$2. R\Lambda = \Lambda R = R;$$

$$R\emptyset = \emptyset R = \emptyset ;$$

$$(RS) T = R (ST)$$

$$3. R (S + T) = RS + RT;$$

$$(S + T)R = SR + TR$$

$$4. R^* = R^* R^* = (R)^* = (R^*)^* = (\Lambda + R)^*$$

$$5. R^* = \Lambda + R + R^2 + \dots R^k + R^{k+1} R^*,$$

$$\forall (k \geq 0)$$

$$R^* = \Lambda + R R^*$$

$$6. (R + S)^* = (R^* + S^*)^* = (R^* S^*)^*$$

$$= (R^* S^*) R^* = R^* (S R^*)^*$$

$$(R + S)^* \neq R^* + S^*$$

$$7. R^* R = R R^*;$$

$$R(S R^*) = (R S)^* R$$

$$8. (R^* S)^* = \Lambda + (R + S)^* S;$$

$$(R^* S)^* = \Lambda + R(R + S)^*$$

$$9. \text{Правило на Ардън: ако } \Lambda \notin S$$

$$R = SR + T \leftrightarrow R = S^* T;$$

$$R = RS + T \leftrightarrow R = TS^*$$

Задача 1. Зададена е азбука Σ , определена от множеството думи $\{0, 1\}$. Съставете множество от думи с дължина 4.

Дадено $\Sigma = \{0, 1\}$.

А) Да се определи Σ^+ при $|\Sigma^+| = 4$.

В) Да се състави множество от символи с дължина 4 и всеки низ да се превърне в число от 16 бройна система.

С) Да се състави конкатенация от двойка низове получени при условие В.

Решение:

А) $\Sigma^* = \{????\}$

В) ????

С) ????

Задача 1. Зададена е азбука Σ , определена от множеството думи $\{0, 1\}$. Съставете множество от думи с дължина 4.

Дадено $\Sigma = \{0, 1\}$.

А) Да се определи Σ^+ при $|\Sigma^+| = 4$

В) Да се състави множество от символи с дължина 4 и всеки низ да се превърне в число от 16 бройна система.

С) Да се състави конкатенация от двойка низове получени при условие В.

Решение:

А) $\Sigma^+ = \{0000, 0001, 0010, 0011, 0100, 0101, 0110, 0111, 1000, 1001, 1010, 1011, 1100, 1101, 1110, 1111\}$

В) 0000 (0), 0001 (1), 0010 (2), 0011 (3),
0100 (4), 0101 (5), 0110 (6), 0111 (7),
1000 (8), 1001 (9), 1010 (A), 1011 (B),
1100 (C), 1101 (D), 1110 (E), 1111 (F)

С) 00, 01, 02, 03, 04, 05, 06, 07, 08, 09, 0A, 0B, 0C, 0D, 0E, 0F
A0, A1, A2, A3, A4, A5, A6, A7, A8, A9, AA, AB, AC, AD, AE, AF ...
F0, F1, F2, F3, F4, F5, F6, F7, F8, F9, FA, FB, FC, FD, FE, FF

Задача 2. Зададени са две множества от низове

$A = \{a, ab, aab, aba, abb\}$ и $B = \{b, ba, bba, bab, baa\}$.

Да се съставят конкатенацията на множествата: $C = A.B$ и $D = B.A$

Първо се записва първият елемент от множество A към който се добавя първи елемент от множество B , следва втори елементи от A и първи от B , докато изброим всички елементи от множество A . Преминаваме към конкатенацията на първи елемент от A с втори елемент от B и т.н.

Решение:

$C = A.B = \{?????\}$

$D = B.A = \{????\}$

Задача 2. Зададени са две множества от низове

$A = \{a, ab, aab, aba, abb\}$ и $B = \{b, ba, bba, bab, baa\}$.

Да се съставят конкатенациите на множествата: $C = A.B$ и $D = B.A$

Първо се записва първият елемент от множество A към който се добавя първи елемент от множество B , следва втори елементи от A и първи от B , докато изброим всички елементи от множество A . Преминаваме към конкатенацията на първи елемент от A с втори елемент от B и т.н.

Решение:

$C = A.B = \{ab, abb, aabb, abab, abbb,$
aba, abba, aabba, ababa, abbba,
abba, abbbba, aabbbba, ababba, abbbba,
abab, abbab, aabbab, ababab, abbbab,
abaa, abbaa, aabbaa, ababaa, abbbbaa}

$D = B.A = \{ba, baa, bbaa, baba, baaa,$
bab, baab, bbaab, babab, baaab,
baab, baaab, bbaaab, babaab, baaaab,
baba, baaba, bbaaba, bababa, baaaba,
babb, baabb, bbaabb, bababb, baaabb}

Задача 3. Дадено е множество $S=\{ab, ba\}$ да се образува S^* (затворена обвивка)

Решение:

Затворена обвивка на множеството S се образува чрез обединяване на множество от множествата низове с дължина 0, 1, 2, Съответно S^0, S^1, S^2 и т.н.

$$S^* = S^0 \cup S^1 \cup S^2 \dots, \text{където}$$

$$S^0 = \{\wedge\}$$

$$S^1 = \{ab, ba\}$$

$$S^2 = \{abab, abba, baab, baba\}$$

Опитайте самостоятелно за домашна работа !!!

$$S^3 = ?$$

$$S^4 = ?$$

$$S^5 = ?$$

Задача 4 . Да се докаже равенството $R(SR)^* = (RS)^*R$

Решение:

1) Нека $w \in R(SR)^*$ е произволен елемент,
който принадлежи на лявата част на равенството.

=> може да се образува низ с дължина $n \geq 0$ от букви (низове):

$$r_i \in R \text{ и } s_i \in S,$$

$$w = r_0 (s_0 r_1)(s_1 r_2) \dots (s_{n-1} r_n)$$

тъй като конкатенацията на низ притежава свойство асоциация, то

$$\Rightarrow w = (r_0 s_0)(r_1 s_1) \dots (r_{n-1} s_{n-1}) r_n$$

$$\Rightarrow w \in (RS)^*R, \text{ т.е. } w \in R(SR)^* \subseteq (RS)^*R$$

2) Нека $w \in (RS)^*R$

$$w = (r_0 s_0)(r_1 s_1) \dots (r_{n-1} s_{n-1}) r_n, \text{ което се представя, като:}$$

$$w = r_0 (s_0 r_1)(s_1 r_2) \dots (s_{n-1} r_n), \text{ т.е. } w \in R(SR)^*$$

$$\Rightarrow (RS)^*R \subseteq R(SR)^*$$

Задача 5 . Да се докаже правилото на Ардън - ако $\Lambda \notin S$, то $R = SR + T \leftrightarrow R = S^*T$

Решение:

Приема се, че $R = SR + T$

1) Да се докаже, че $R = S^*T$

от $R = SR + T$

$$\Rightarrow R = S(SR + T) + T$$

$$\Rightarrow S^2R + ST + T$$

$$\Rightarrow S^2(SR + T) + ST + T$$

$$\Rightarrow S^3R + S^2T + ST + T$$

$$R = S^{k+1}R + (S^kT + S^{k-1}T + \dots + ST + T), \text{ за всяко } k \geq 0$$

Нека $x \in R$ и $|x| = l$

Щом равенството l , е вярно за всяко $k \geq 0$, то е вярно и за $k=l$

$$\Rightarrow R = S^{l+1}R + (S^lT + S^{l-1}T + \dots + ST + T) \text{ по определение}$$

$$S^*T = (S^lT + S^{l-1}T + \dots + ST + T), \text{ за низ с дължина } l$$

Щом $\Lambda \notin S$ всяка дума от $S^{l+1}R$ трябва да има поне $l+1$ символа

$$\Rightarrow x \notin S^{l+1}R, \text{ но } x \in R \Rightarrow x \in (S^lT + S^{l-1}T + \dots + ST + T)$$

$$\Rightarrow x \in S^*T$$

$$\Rightarrow R \subseteq S^*T$$

Задача 5 . Да се докаже правилото на Ардън - ако $\Lambda \notin S$, то $R = SR + T \leftrightarrow R = S^*T$

Решение - продължение

2) Да се докаже, че $S^*T \subseteq R$

Нека $x \in S^*T$. Тогава съществува $i \geq 0$, такава, че $x \in S^iT$

От равенство 1) $R = S^{i+1} + (S^iT + S^{i-1}T + \dots + ST + T)$

$\Rightarrow R \subseteq S^iT$

$\Rightarrow x \in R$

Тема 2. Подредени множества и низове.

Операции с низове. Регулярни множества и изрази

6. Работа с регулярни изрази. Приложение на регулярните изрази

6.1. Регулярни изрази в C#

Стандартните класове за работа със символни низове в C#:

- String;
- StringBuilder.

Предназначението им е за прилагане на операции на стрингове, като *търсене, вмъкване, извличане, слепване и разделяне на низове, и др.*

Прилагането им е ограничено, при по-сложно търсене.

Например да се намери всеки израз от типа "a/(b+c)" и да се замени с израз като, "(b-c)*a".

Решението е да се използват възможностите на Net Framework за обработка на символни низове, основани на регулярните изрази. Именовано пространство RegularExpression, съдържа набор от класове, за работата с регулярни изрази.

Регулярният израз е символен низ, конструиран по специални синтактични правила. Чрез него се задава шаблон, с който се извършва проверка дали даден низ удовлетворява шаблона.

Приложението им в практиката е свързано с **търсене, валидация и заместване.**

Тема 2. Подредени множества и низове.

Операции с низове. Регулярни множества и изрази

Синтаксис на регулярните изрази в C#

Примери за регулярни изрази:

```
Regex pattern = new Regex ("abc");
```

Литералът "abc", задаващ шаблон за търсене, при извикване на метод, намира едно или повече срещания на подниза "abc" в даден низ.

Регулярните изрази се делят на две основни групи:

- ☐ **литерали** - символи и поднизове в регулярния израз, които се възприемат от апарата за обработка буквално (както в примера по-горе), по начина по който са написани;
- ☐ **метасимволи** - символи със специално значение.

Тема 2. Подредени множества и низове.

Операции с низове. Регулярни множества и изрази

Символ	Символи, използвани в регулярните изрази
	Интерпретация
	Категория: escape-последователности
\b	При използването му в квадратни скоби, съответства на символа "обратна наклонена черта", с код - \u0008
\t	Съответства на символа табулация, с код - \u0009
\r	Съответства на символа Carriage Return, с код - \u000D
\n	Съответства на символ за нов ред, с код - \u000A
\\	Съответства на символ \
*	Съответства на символ *
\+	Съответства на символ +
\e	Съответства на символа escape, с код - \u001B
\040	Съответства на ASCII символ, зададен чрез осмичен код
\x20	Съответства на ASCII символ, зададен с 2 цифри, в шестнадесетичен код
\u0020	Съответства на Unicode символ, зададен с 4 цифри, в шестнадесетичен код

	Категория: подмножества (класове) от символи
.	Съответства на един произволен символ, с изключение на символа край на ред, в Windows това е \r\n. Например, "a.b" съвпада с "aab," "a2b," и така нататък но не на "ab" или "auxb". Чрез (. s) може да обозначим произволен символ, включително „празно пространство.
[yes]	Съответства на кой да е символ от множеството,
	зададено в квадратните скоби, тоест квадратните скоби в езика на регулярните изрази указват клас от символи – в случая това са всички указани 3 символа. Също [0-9] означава клас, включващ числата от 0 до 9.
[^aeiou]	Отрицание. Съответства на кой да е символ, с изключение на символите, зададени в квадратните скоби след ^. Например, шаблонът example не допуска цифри: Regex example = new Regex("[^0-9]");
[0-9a-fA-F]	Задава диапазона на символите. Така, [0-9] задава коя да е цифра, аналогично на шаблона [0123456789].
\p{name}	Съответства на кой да е символ, зададен чрез множеството с име „name”, например, \l задава множеството от малките латински букви.
\P{name}	Отрицание. Голямата буква винаги задава отрицание на множеството, с име зададено в скобите
\w	Включва множеството от големи и малки символи на латиница, цифри и знак за подчертаване, тоест еквивалентен на [A-Za-z_0-9]. \W – означава всички символи, които не принадлежат на горния клас. Например, \w* в един израз означава произволна последователност (може и с нулева дължина) от букви, цифри и знак за подчертаване.

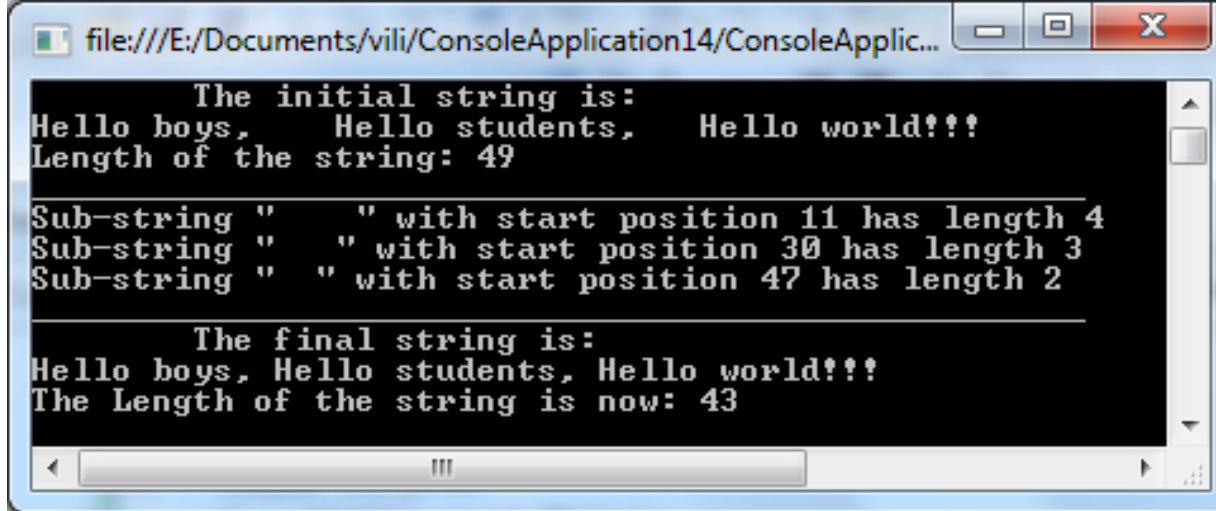
\s	Съответства на всеки символ за обозначаване на „празно пространство“ (интервал, табулация, символ за нов ред и т.н). \S – Означава всички символи, които не са празно пространство.
\d	Съответства на кой да е символ от множеството на цифрите, \D – всички символи, които не са десетични цифри. Например, чрез шаблони [^\d\s] и: [\D\S] може да търсим символ, който не е цифра, нито символ „празно пространство“.
Категория: Операции (модификатори)	
*	Итерация. Задава нула или повече съответствия; например \w* - обозначава всички символни низове, съставени от големи и малки латински букви, цифри и знак за подчетаване; [01]* обозначава всички символни низове, съставени от цифрите 0 или 1, включително празния низ; Аналогично, на {0,}.
+	Положителна итерация. Задава 1 или повече срещания на предходния символ, например, (abc)+ указва едно или повече срещания на низа abc. А [0-9]+ указва, че целият клас символи [0-9] трябва да се среща последователно 1 или повече пъти. Regex age = new Regex ("Age [1-9]+"); Шаблон age ще намери съвпадения на всички поднизове (литерали), които започват с думата "Age ", следвана от поне една десетична цифра в диапазона от 1 до 9.

	обозначава алтернативни възможности (метасимвол). Например (Ана Яна) означава, търсене за съвпадение за Ана или Яна.
?	Задава нула или едно съответствия, например, \w? обозначава голема и малка латинска буква, или цифра, или знак за подчертаване или празен низ. Аналогично на {0,1}.
{n}	Указваме, че предхождащия символ ще се повтаря точно n пъти, например, \w{2} – указва точно 2 символа (латински букви, цифри или знак за подчертаване); [0-9]{3} обозначава последователност от точно 3 цифри.
{n,}	Задава, най-малко n съответствия; например, например, (abc){2,} – търсим най-малко 2 срещания на низа abc. [0-9]{3,} обозначава последователност от поне 3 цифри.
{n,m}	Задава, най-малко n съответствия, но не повече от m; например, (abc){2,5} – търсим най-малко 2, но не-повече от 5 срещания на низа abc.
^	Задава началото на низ, например "^x" - низ, който започва с "x".
\$	Задава края на низ, например "x\$" обозначава низ, който има "x" в края си. Така шаблона "^x\$" съвпада с "x" - низ, състоящ се от единствен символ.
(група), (?<име> група)	Задава група в регулярния израз (без име или с име). Групите се използват за логическо отделяне на части от регулярния израз и могат да имат имена, например: \s*(?<name>\w+)\s*=\s*(\d+)

Пример: Нека е даден низ test = "Hello boys, Hello students, Hello world!!! ". Да се изведат поднизовете в низа test, съдържащи повече от една празна позиция (в случая ще са 3 на брой). Да се организира заместването им с една празна позиция, т.е. между думите в резултатния низ test **не трябва да има повече от една ипация.**

Решение:

```
using System;
using System.Text.RegularExpressions;
class Test_Regular_Expressions {
static void Main() {
string test = "Hello boys, Hello students, Hello world!!! ";
Console.WriteLine("\tThe initial string is: \n{0}", test);
Console.WriteLine("Length of the string: {0}", test.Length);
string separator = "_____";
Console.WriteLine(separator);
Regex r = new Regex(@"\s{2,}");
/*дефинираме Шаблон r, с който в последствие ще търсим 2 и повече ипации в текста test */
Match match = r.Match(test);
//търсим съвпадение със шаблона
while (match.Success) {
/*Ако е намерен подстринг, отговарящ на шаблона, то извеждаме подстринга, индекса му
стринг test и в дължината му */
Console.WriteLine("Sub-string \"{0}\" with start position {1} has length {2}", match, match.Index, match.Length);
test = Regex.Replace(test, @"\s{2,}", " ");
//Минимум 2 броя празни позиции в низ test се заместват с една
match = match.NextMatch();
//търсим следващо съвпадение със шаблона
}
Console.WriteLine(separator);
Console.WriteLine("\tThe final string is: \n{0}", test);
Console.WriteLine("The Length of the string is now: {0}", test.Length); }
}
```

A screenshot of a Windows console application window. The title bar shows the file path: file:///E:/Documents/vili/ConsoleApplication14/ConsoleApplic... The console output is as follows:

```
The initial string is:  
Hello boys,    Hello students,    Hello world!!!  
Length of the string: 49  
  
Sub-string "    " with start position 11 has length 4  
Sub-string "    " with start position 30 has length 3  
Sub-string "    " with start position 47 has length 2  
  
The final string is:  
Hello boys, Hello students, Hello world!!!  
The Length of the string is now: 43
```

Използвани са методите на класовете `Regex` и `Match` за работа с регулярни изрази. Те са разположени в именованото пространство `System.Text.RegularExpressions`.

Чрез клас `Regex` се дефинира регулярен израз (шаблон за търсене), в случая с име „r“. Той намира всички поднизове в низ `test`, съдържащи 2 и повече празни позиции. При дефиниране на шаблона са използвани метасимволите:

- `\s` - за указване на празно пространство
- `{2,}` - за указване на поне 2 повторения на предходния символ, а именно „празно пространство“.

Чрез методи `Match()` на клас `Regex` и `NextMatch()` на клас `Match` се откриват първото и следващите успешни съвпадения с шаблона. При клас `Match` се използват свойството `Success`, което връща булева стойност и показва дали съвпадението е успешно, свойството `Index`, което извежда позицията в низа, от която започва съвпадението (броенето е от 0) и свойството `Length`, което извежда дължината на съвпадението.

Тема 2. Подредени множества и низове.

Операции с низове. Регулярни множества и изрази

Основни класове за работа с регулярни изрази в .NET Framework

А. Клас **Regex**

Класът **Regex** представя един неизменим (константен) шаблон. Той е задължителен при работа с регулярни изрази. Методите му спомагат за извършване на операции с РИ: търсене, заместване, разделяне на низ по регулярен израз и др.

Класът **Regex** се използва, както чрез свой обект, инициализиран с даден шаблон, така и чрез статичните си методи. При статичните методи, които **Regex** предлага, всеки път трябва да се подава като параметър, необходимия шаблон.

Пример за прилагане на алтернативните възможности, използващи клас **Regex**:

```
string test = "Hello boys,  Hello students,  Hello world!!! ";
string some_pattern = "\\s{2,}";
// Instance Regex
Regex r = new Regex(some_pattern);
Match match1 = r.Match(text);
// Static Regex
Match match2 = Regex.Match(text, some_pattern);
```

Основни методи на клас **Regex**:

Метод **Match** извършва едно търсене в низа, за съвпадение с шаблона. Като параметър се подава низа, в който се извършва търсене (в примера **test**). Резултатът е обект от клас **Match**, както при успешно, така и при неуспешно търсене. За да се провери дали има валиден резултат, се използва свойството **Success** на класа **Match**. Действието му се прекратява при първото намерено съвпадение. Чрез метода се намират всички поднизове отговарящи на шаблона. Резултат е обект от тип **MatchCollection**, представляващ колекция от обекти от тип **Match**.

Метод **NextMatch** стартира ново търсене, започвайки от мястото до което е стигнало предното търсене. Метод **Split** е една по-ефективна реализация на метод **Split** на клас **String**. Разделя низа на елементи. Когато шаблона е много сложен, метода при клас **Regex** е по-ефективен от аналогичния при клас **String**.

В. Клас Match

Класът **Match** представя съвпадение при търсене с РИ, ясно стана, че обект от този клас се създава при извикване на метод **Match** на клас **Regex**. Свойствата на класа **Match** са от важно значение те дават информация за намереното съвпадение: съдържание, дължина, начална позиция в низа и други.

Метод **NextMatch()** на класа **Match**, позволява да се премине към следващото намерено съвпадение.

Свойствата **Index**, **Length** и **Value** описват намерения подстринг – индекса на подниза в търсения низ, дължината на низа и нейната стойност.

Тема 2. Подредени множества и низове.

Операции с низове. Регулярни множества и изрази

C. Клас MatchCollection

Обекти на класове Match и MatchCollection се създават автоматично при извикване съответно на методи Match и Matches на клас Regex. Класът MatchCollection съдържа списък от всички съвпадения, получени при търсенето с шаблона върху изрази.

Колекцията MatchCollection, както и всички колекции, позволява да се получи достъп към всеки неин елемент, който е обект от тип Match. Може разбира се да се организира цикъл foreach за последователен достъп до всеки елемент на колекцията.

D. Класовете Group и Capture

Клас Group се явява наследник на клас Capture и едновременно – родител на клас Match. От своя родител клас Group, той наследява свойствата Index, Length и Value, които предава на своя наследник Match. Класовете Group и Capture, както и съответните класове за колекции GroupCollection и CaptureCollection, са свързани с механизма на групите в регулярните изрази. Техните методи дават редица възможности за работа с тези групи.

Колекция GroupCollection се връща като резултат от извикването на свойството Group на обект от тип Match. Разполагайки с тази колекция, можем да се достигне до всеки неин обект, който е от тип Group.

Тема 2. Подредени множества и низове.

Операции с низове. Регулярни множества и изрази

Е. Клас MatchEvaluator

Делегатът MatchEvaluator се използва при операциите за заместване, като има възможност да прилагане на дефинираната в него потребителска функционалност върху всяко намерено съвпадение и за заместване с получения резултат.

Тип RegexOptions

Изброяването RegexOptions дефинира различни константи, които се използват за указване на различни опции, свързани с процеса на търсене с регулярни изрази.

Е. Клас RegexCompilationInfo

Клас RegexCompilationInfo се използва в процеса на компилиране на РИ в самостоятелни ансамбли. При работа със сложни и големи текстове е полезно предварително да се компилира използвания в процеса на търсене регулярен израз. В този случай се създава обект от клас RegexCompilationInfo, на който се предава информация за регулярния израз, подлежащ на компилация и за мястото, където да се помести компилираната програма. Допълнително, в такива ситуации следва да се включи опция Compiled.

Тема 2. Подредени множества и низове.

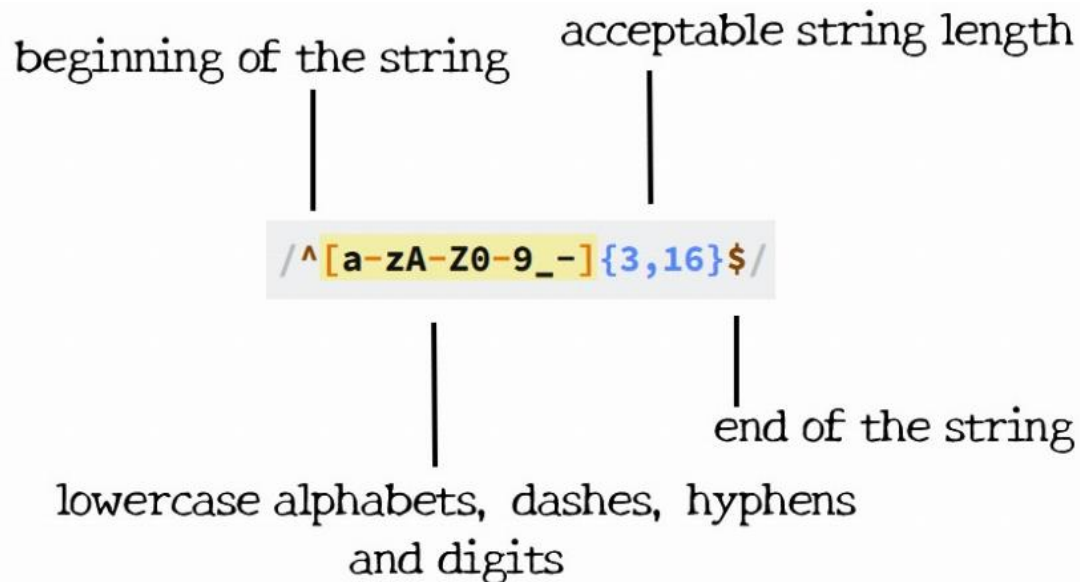
Операции с низове. Регулярни множества и изрази

7. Някои често използвани РИ:

- `username`
- `password`
- шеснадесетично число
- `email`
- `URL`
- `IP address`
- `HTML tag`
- `дата`

Разделителите, използвани в регулярните изрази, са наклонени черти: /. Всеки шаблон започва и завършва с разделител. Ако в регулярен израз се появи наклонена черта, трябва да се екранира с обратна наклонена черта по следния начин: \.

- **username**



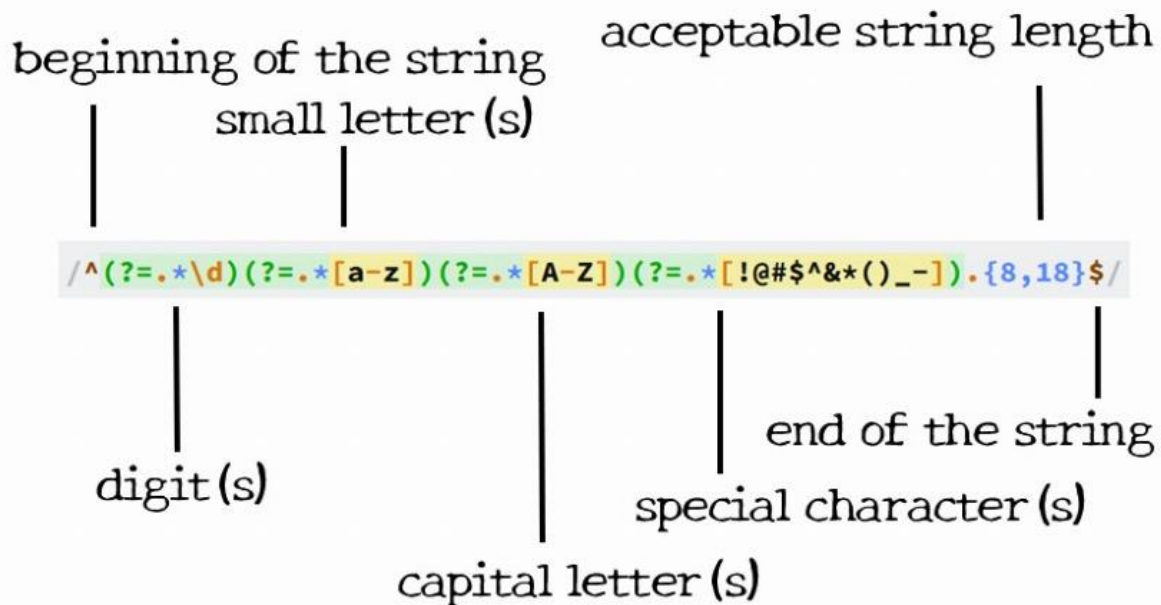
Описание:

Анализаторът намира началото на низ (^), последван от всяка малка буква (a-z), главна буква (A-Z), число (0-9), долна черта или тире. {3,16}- означава поне 3 от посочените символи, но не повече от 16. За край на низа (\$).

Низ, който съвпада със шаблона - my-USER_n4m3

Низ, който не съвпада със шалона - th1s1s-wayt00_l0ngt0beausername (твърде е дълъг)

- password



Описание:

Обикновено, когато се създава парола тя трябва да включва: една цифра, поне една малка буква, поне една главна буква и поне един специален знак. Често се прави проверка за дължината, например да е между 8 и 18 знака. Именно РИ проверява всички тези условия.

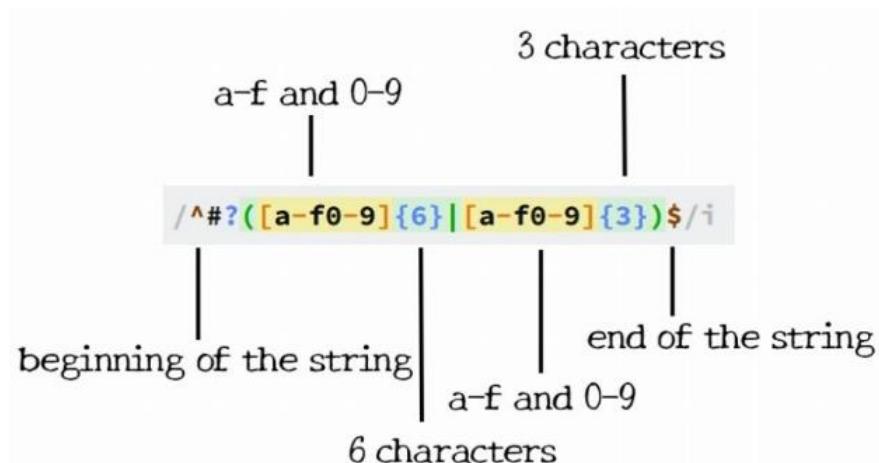
В конкретния случай РИ е разделен на пет отделни части. Първите четири части са положителни изрази, при които в скоби, с префикс `?=..` е записано `(?=.*\d)`. Положителните изрази за съвпадение навсякъде в низа; не е необходимо да се появяват в ред.

Изразът `(?=.*\d)` търси всякакви цифри навсякъде в низа. По същият начин `(?=.*[a-z])` търси всички малки букви и `(?=.*[A-Z])` търси главни букви. Ако беше използван изразът `(?=.*[a-zA-Z])` в РИ, това би означавало, че наличието на малка или главна буква би дало положителен резултат, но тук целта е, РИ да включва и малки и главни букви.

*Низ, който съответства на шаблона - **Myр^ssw0rd***

*Низ, който не съвпада със шаблона - **Myр^ssword** (не съдържа никакви цифри)*

■ шеснадесетично число



Описание:

Анализаторът трябва да намери началото на низа (^). След това знакът # не е задължителен, защото е последван от ?. Въпросителният знак казва на анализатора, че предходният знак – в този случай знак за # – не е задължителен, но ако е там трябва да го улови.

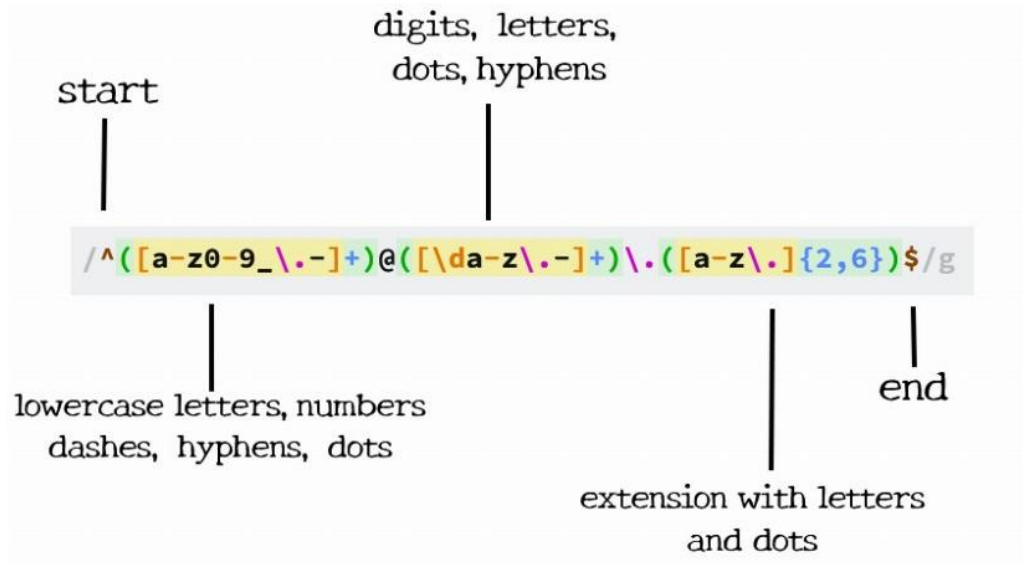
След това, вътре в първата група (първата група от скоби), може да има две различни ситуации. Първата е всяка малка буква между а и f или число шест пъти. Символът | е равносителен на ИЛИ, т.е. може да има три малки букви между а и f или цифри. В края, има символ за край на низ (\$). Използван е флаг за чувствителност към регистъра, като се добави i в края на израза. Това ще позволи съпоставяне на #ffffff и #FFFFFF.

Причината, за да се поставят шестте знака към първата група, а не след, е, че анализаторът ще улови шеснадесетична стойност като #ffffff. Ако трите знака бяха първи, анализаторът щеше да вземе само, #fffa не другите три "f" - а.

Низ, който съответства с шаблона - #a3c113

Низ, който не съвпада със шаблона - #4d82h4 (съдържа буквата h)

- email



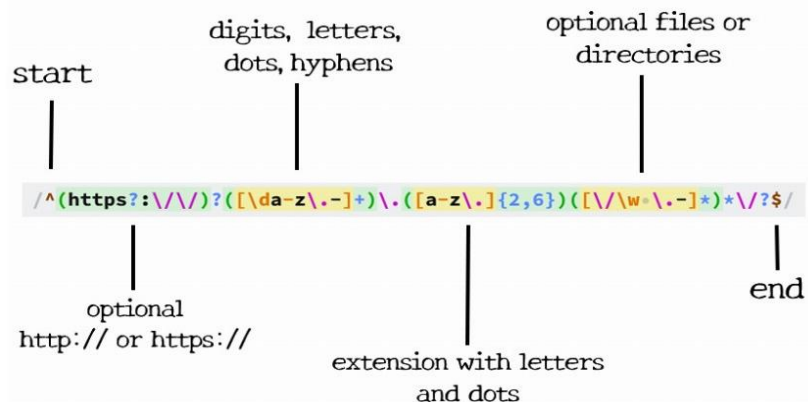
Описание:

Анализаторът трябва да намери началото на низа (^). Вътре в първата група се съпоставя една или повече малки букви, цифри, долна черта, точки или тирета. Точката е избегната, защото неекранирана точка означава произволен знак. Веднага след това трябва да има @. Следва името на домейна, което трябва да се състои от една или повече малки букви, цифри, долна черта, точки или тирета. След това друга точка, като разширението е от 2 до 63 букви или точки. Има от 2 до 63 поради специфичните за страната TLD (.ny.us или .co.uk) и, разбира се, новите правила позволяват по-дълги TLD. И накрая, има край на низа (\$).

Низ, който съответства на шаблона: john@doe.com

Низ, който не съвпада със шаблона: john@doe (лунсва TLD)

■ URL



Описание:

РИ е подобен на РИ за mail-a, като се вгради крайната част на РИ за mail-a, между "https://" и файлова структура в края. Търсенето започва с началото на реда с каретката.

Първата група за улавяне не е задължителна. Позволява URL адресът да започва с "http://", "https://" или нито едно от тях. Въпросителният знак след s, служи за разрешаване на URL адреси, които имат http или https . В края на групата се добавя ? И така тя става незадължителна.

Следва името на домейна: една или повече цифри, букви, точки или тирета, последвани от друга точка, след това две до шест букви или точки.

Следващият раздел е за незадължителните файлове и директории. Вътре в групата се съпоставят произволен брой наклонени черти, букви, цифри, долни черти, интервали, точки или тирета. Група може да бъде съпоставена толкова пъти, колкото е необходимо. Това позволява множество директории да бъдат съпоставени заедно с файл в края. Използването на звезда вместо въпросителен знак, позволява нула или повече, а не нула или едно.

Следва крайна наклонена черта, която може да е незадължителна. Завършва се със символ за край на реда.

*Низ, който съответства на шаблона: **https://tutsplus.com/about***

*Низ, който не съвпада със шаблона: **http://google.com/some/file!.html** (съдържа удивителен знак)*

▪ **IP Address**

```
/^(?:(?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.){3} (?:(?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)$/
```

Първата група за улавяне не е уловена група, защото ?: е поставена вътре, което указва на анализаторът да не прихваща тази група. Повторение три пъти на група посочено с {3} в края на групата. Тя съдържа друга група, подгрупа и точка. Анализаторът търси съвпадение в подгрупата и след това точката, за да продължи напред.

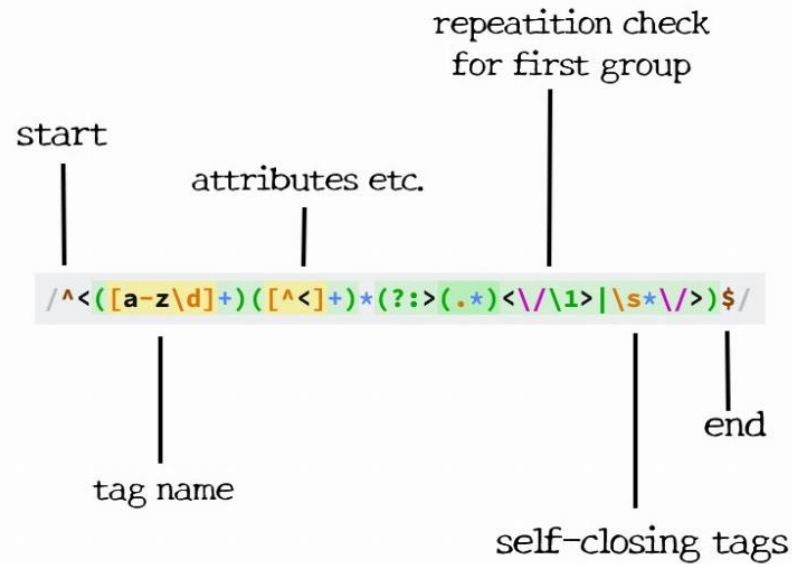
Подгрупата също е друга група без улавяне. Това е набор от знаци, които заедно описват числата между 0 и 255 (това, което е в скоби): низът "25", последван от число между 0 и 5; или низ "2" и число между 0 и 4 и произволно число; или незадължителна нула или единица, последвана от две числа, като второто не е задължително.

След като съпоставим три от тях, това е към следващата група без улавяне. В нея низът "25" е последван от число между 0 и 5; или низът "2" с число между 0 и 4 и друго число в края; или незадължителна нула или единица, последвана от две числа, като второто не е задължително. В края на РИ е поставен символ за край на низ.

Низ, който съответства на шаблона: 73.60.124.136

Низ, който не съвпада със шаблона: 256.60.124.136 (всички части трябва да са под 255)

■ HTML tag



Описание:

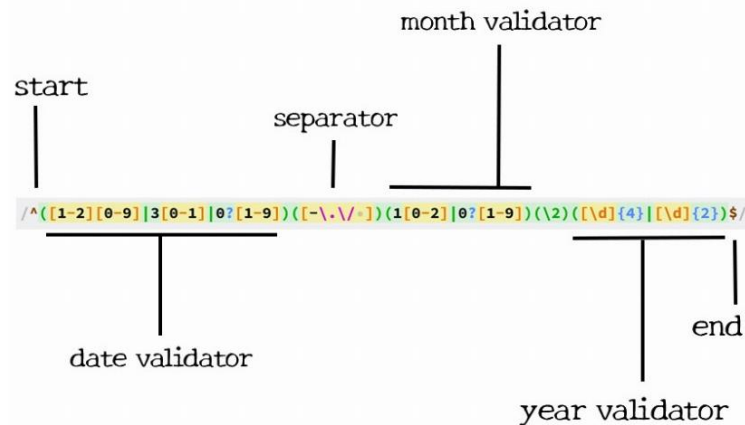
Анализаторът трябва да намери началото на низа (^). Първо е името на етикета, трябва да съдържа една или повече букви. Това е първата група за улавяне и е полезна, когато трябва да се хване затварящия таг. Следващото нещо са атрибутите на етикета. Това е произволен знак, освен знака по-голямо от (>). Тъй като това не е задължително, но трябва да се съпостави повече от един символ, се използва символа звезда. Знакът плюс съставлява атрибута и стойността, а звездата казва колко атрибута желаем.

Следва третата група без улавяне, съдържа или знак за по-голямо от, малко съдържание и затварящ таг; или няколко интервала, наклонена черта и знак за по-голямо от. Първата опция търси знак за по-голямо от, последван от произволен брой знаци и затварящ таг. \1 подобно на първата група за заснемане. В този случай това е името на маркера. Ако това не може да бъде съпоставено, се търси самозатварящ се етикет (като етикет `img`, `br` или `hr`). Като в случая трябва да има един или повече интервали, последвани от `/>`. Регулярният израз завършва с края на реда.

*Низ, който съответства на шаблона: **<h1 data-tip="Some Hint">Голямо заглавие</h1>***

*Низ, който не съвпада със шаблона: **<h1 data-tip="Some Hint">Голямо заглавие</h2>** (отварящият и затварящият тагове не съвпадат).*

■ Дата



Описание:

Датите се изписват в различни формати тук е разгледан формата ДД/ММ/ГГГГ или ДД/ММ/ГГ.

Датите в месеца са от 1 до максимум 31. Начина на записване може да е 02 или 2 за деня от месеца. Това се покрива от израза. Ако първата цифра е 1 или 2, втората цифра може да между 0 и 9. Ако първата цифра е 3, втората цифра е разрешено да бъде само 0 или 1. За разделител се използва само тире, точка, интервал или наклонена черта. Това се поставя в група за улавяне, за да може да се провери дали се използва един и същ разделител между стойността на месеца и годината. Месецът може да бъде само до 12, така че е позволено втората цифра да бъде само 0, 1 или 2, ако първата цифра е 1. Няма ограничение за номера на годината. Може да бъде произволно четирицифрено число като 1508 или 9999. Позволено е годината да бъде написана с две цифри, в случай че някой иска да напише датата като 11/09/91.

РИ е за дати, които следват формата ДД/ММ/ГГГГ. **Опитайте да го промените за формата на датата ММ/ДД/ГГГГ.**

Още нещо, което трябва да се отбележи е, че РИ ще счита 31.02.1991 г. за валидна дата, което не е коректно, тъй като февруари има най-много 29 дни. Като РИ трябва се промени така, че, броят на дните през февруари да не надвишава 28 за невисокосни години и 29 за високосни. Това ще направи РИ сложен. Много по-практично е да се използват библиотеки за валидиране на дати за тези крайни случаи!!!!

№	Описание на регулярен израз	Пример	Регулярен израз
1	Едноцифрено число от 0 до 9	0, 1, 2, 3 ...	
2	Двучифрени числа от 10 до 99	10, 20, 99	
3	Едноцифрено число в интервала 0 - 1	0, 1	
4	Множество символи, които могат включващи само цифри 0 или 1	01, 11, 000111 ...	
5	Трицифрени числа в интервала от 100 до 999	100, 999, 800 ...	
6	Двучифрени числа в интервала от 0 до 99	0, 10, 99	
7	Последователност от цифри, започваща с 088, следвана от 7 символа, всеки от които е в интервала от 0 до 9	0881234567, 0880000000, ...	
8	Последователност от цифри, започваща с 025, следвана от 6 символа, всеки от които е в интервала от 0 до 9	025123456, 025000000 ...	
9	Единичен символ включващ малки и големи латински букви, цифри от 0 до 9 и символ „_“	0, 1, a, A, Z ...	
10	Произволни думи, състоящи се от 1 до 6 символа, включващи малки и големи латински букви и цифри от 0 до 9, като завършва задължително с последен символ „?“	123456?, 1?, 121?	
11	Трицифрени числа в интервала от 1 до 999	1, 42, 455, 120, 999	
12	Трицифрени числа в интервала от 100 до 999, с фиксиран брой на цифрите	100, 222, 999 ...	
13	Трибуквени думи, първият символ да не е “c” следващите два да са “ab”	kab, asa ...	
14	Израз включващ думи, започващи с “f” или „с“ или “m” след които има сричката “at” а в края има „“	fat. cat. mat.	
15	От три до седем символа, които са в интервала a-z A-Z 0-9_	aZ_, 01_asZ, ...	

№	Описание на регулярен израз	Пример	Примерно решение Регулярен израз
1	Едноцифрено число от 0 до 9	0, 1, 2, 3 ...	[0-9]
2	Двучифрени числа от 10 до 99	10, 20, 99	[1-9][0-9]
3	Едноцифрено число в интервала 0 - 1	0, 1	[0-1]
4	Множество символи, които могат включващи само цифри 0 или 1	01, 11, 000111 ...	[0-1]+
5	Трицифрени числа в интервала от 100 до 999	100, 999, 800 ...	[1-9][0-9][0-9]
6	Двучифрени числа в интервала от 0 до 99	0, 10, 99	[0-9]{1,2}
7	Последователност от цифри, започваща с 088, следвана от 7 символа, всеки от които е в интервала от 0 до 9	0881234567, 0880000000, ...	088[0-9]{7}
8	Последователност от цифри, започваща с 025, следвана от 6 символа, всеки от които е в интервала от 0 до 9	025123456, 025000000 ...	025[0-9]{6}
9	Единичен символ включващ малки и големи латински букви, цифри от 0 до 9 и символ „_“	0, 1, a, A, Z ...	[a-zA-Z0-9_]
10	Произволни думи, състоящи се от 1 до 6 символа, включващи малки и големи латински букви и цифри от 0 до 9, като завършва задължително с последен символ „?“	123456?, 1?, 121?	\w{1,6}[?]
11	Трицифрени числа в интервала от 1 до 999	1, 42, 455, 120, 999	[0-9]{1,3}\$
12	Трицифрени числа в интервала от 100 до 999, с фиксиран брой на цифрите	100, 222, 999 ...	[0-9]{3}
13	Трибуквени думи, първият символ да не е “с” следващите два да са “ab”	kab, asa ...	[^c]ab
14	Израз включващ думи, започващи с “f” или „с“ или “m” след които има сричката “at” а в края има „.“	fat. cat. mat.	(f c m)at\.
15	От три до седем символа, които са в интервала a-z A-Z 0-9_	aZ_, 01_asZ, ...	[a-zA-Z0-9_]{3,7}\$